Deep Learning para Texto y Secuencias

Día 2: Embeddings, RNN y GRU

Curso introductorio para TICs y Seguridad

Dr. Rubén Hernández Torres

20 de octubre de 2026

Objetivos

- Comprender cómo representar texto numéricamente.
- Explicar la función de los embeddings.
- Comprender el estado oculto de una RNN.
- Identificar las ventajas de LSTM y GRU.
- Implementar un clasificador de secuencias.

¿Cómo procesar texto con una red neuronal?

Las redes neuronales operan sobre tensores, no directamente sobre cadenas de caracteres.

Ejemplo de mensaje:

```
verify your account
```

Necesitamos transformar los símbolos en representaciones numéricas.

Aplicaciones

Clasificación de phishing, análisis de logs, detección de spam y análisis de comandos.

Tokenización

La tokenización divide una entrada en unidades discretas llamadas tokens.

- Palabras.
- Caracteres.
- Subpalabras.
- Identificadores de eventos.

Ejemplo:

$$[\text{verify}, \text{your}, \text{account}]$$

Vocabulario ilustrativo:

$$\text{verify} \mapsto 1, \quad \text{your} \mapsto 2, \quad \text{account} \mapsto 3$$

Los índices identifican tokens; no representan relaciones aritméticas.

Representación one-hot

Para un vocabulario de tamaño V:

$$\mathbf{x}_i \in \mathbb{R}^V$$

Cada token tiene un vector con una única componente igual a uno.

$$\text{verify} = [1, 0, 0, 0]$$

$$\text{account} = [0, 1, 0, 0]$$

Limitaciones:

- Alta dimensionalidad.
- No representa similitud semántica.
- Todos los tokens diferentes son ortogonales.

¿Qué es un embedding?

Un embedding transforma un token discreto en un vector denso aprendible.

$$E \in \mathbb{R}^{V \times d}$$

Donde:

- V: tamaño del vocabulario.
- d: dimensión del embedding.
- E: matriz de parámetros.

Para el token i:

$$\mathbf{z}_i = E_{i,:}$$

Habitualmente $d \ll V$.

Relación con one-hot

Sea $\mathbf{x}_i$ el vector one-hot del token i.
Entonces:

$$\mathbf{z}_i = E^T \mathbf{x}_i$$

Esta multiplicación selecciona exactamente una fila de la matriz de embeddings.

Idea importante

`nn.Embedding` implementa una búsqueda por índice, sin construir explícitamente el vector one-hot.

¿Cómo se aprenden los embeddings?

La matriz se inicializa y sus parámetros se ajustan durante el entrenamiento.

$$E^{(k+1)} = E^{(k)} - \eta \nabla_E \mathcal{L}$$

- Los embeddings son parámetros aprendibles.
- Su geometría depende de la tarea.
- La cercanía entre tokens no está garantizada.

Pregunta: ¿Cómo podría cambiar la representación de la palabra `account` en un clasificador de phishing?

Secuencias de longitud variable

Los mensajes pueden contener diferentes cantidades de tokens.

$$[1, 2, 3, 4]$$

$$[5, 6, 0, 0]$$

El padding permite formar lotes de longitud uniforme.

Precaución

`padding_idx` evita actualizar el embedding de relleno, pero no impide que otras operaciones procesen el padding.

Tratamiento correcto del padding

Si se utilizan secuencias rellenadas, es necesario considerar sus longitudes reales.
Alternativas:

- `pack_padded_sequence`.
- Pooling con máscaras.
- Truncamiento y longitudes controladas.

Importante

Tomar el último estado de una GRU sobre secuencias con padding puede incorporar información del relleno.

¿Por qué necesitamos modelos secuenciales?

En muchas aplicaciones el orden importa.

Ejemplo de eventos:

$$\text{login} \rightarrow \text{failed} \rightarrow \text{failed} \rightarrow \text{success}$$

Una representación basada solamente en conteos pierde el orden temporal.
Las redes recurrentes mantienen un estado que evoluciona con la secuencia.

Red neuronal recurrente

Una RNN actualiza su estado oculto:

$$\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b})$$

- $\mathbf{x}_t$: entrada actual.
- $\mathbf{h}_{t-1}$: estado anterior.
- $\mathbf{h}_t$: nuevo estado.

Interpretación

La RNN puede verse como un sistema dinámico discreto no lineal.

Backpropagation Through Time

Para entrenar una RNN se aplica retropropagación a través del tiempo.

$$h_0 \to h_1 \to h_2 \to \cdots \to h_T$$

El gradiente atraviesa las transformaciones recurrentes.

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

Esto puede producir:

- Vanishing gradient.
- Exploding gradient.

LSTM y GRU

Las arquitecturas con compuertas facilitan el manejo de dependencias temporales.

LSTM

- Estado oculto.
- Estado de celda.
- Compuertas de entrada, olvido y salida.

GRU

- Estado oculto.
- Compuerta de actualización.
- Compuerta de reinicio.
- Menos parámetros habitualmente.

No existe una arquitectura universalmente superior.

Actualización de estado en GRU

Una GRU combina información anterior con un estado candidato.
Una formulación conceptual es:

$$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \widetilde{\mathbf{h}}_t$$

La compuerta controla la proporción de información que se conserva o incorpora.

La implementación de PyTorch utiliza una convención y formulación propias.

Evaluación en seguridad

En detección de mensajes sospechosos:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2\frac{PR}{P + R}$$

- Falsos positivos: alertas innecesarias.
- Falsos negativos: amenazas no detectadas.

Práctica propuesta

Clasificación de mensajes
Actividades:

1. Construir un vocabulario.
2. Convertir mensajes en índices.
3. Crear una capa de embeddings.
4. Entrenar un clasificador GRU.
5. Evaluar sobre mensajes nuevos.

Experimento: modificar la dimensión del embedding y el tamaño del estado oculto.

Discusión

¿Mejorar la capacidad del modelo garantiza una mejor generalización?

Conclusiones

- El texto requiere representación numérica.
- Los embeddings son vectores aprendibles.
- Las RNN incorporan dependencias temporales.
- Las GRU utilizan mecanismos de memoria.
- La evaluación debe considerar el costo de los errores.

Siguiente sesión

¿Cómo procesar secuencias mediante mecanismos de atención y aprovechar modelos preentrenados?